

Conversion Prediction — First-look Baseline (v1)

Uncle May's ML

2026-05-02

Table of contents

1	1. Load + inspect	2
2	2. Clean (drop leakage, cap outlier, type-coerce)	4
3	3. Recipes	4
4	4. Models	5
5	5. Cross-validation + fit	6
5.1	5a. Manual per-model fits	6
6	6. Headline metrics table	7
7	7. Variable importance	8
8	8. Summary	10
8.1	Best model	11
8.2	SMOTE verdict	11
8.3	Consensus top features (top-10 across LR, RF, XGB)	12
8.4	Recommendations	12

```
suppressPackageStartupMessages({  
  library(arrow)  
  library(dplyr)  
  library(tidyr)  
  library(tibble)  
  library(stringr)  
  library(lubridate)  
  library(purrr)  
  library(ggplot2)  
  library(scales)  
  library(knitr)  
  
  library(tidymodels)  
  library(workflowsets)  
  library(themis)  
  library(vip)  
  library(hardhat)  
})
```

```

set.seed(42)
# Note: tidymodels_prefer() omitted to dodge a fastmap version conflict on
# this machine. We disambiguate any clashing function names (filter, select)
# inline via package::fun where needed.

# Reusable feature-prep + recipe builder.
source("_feature_prep.R")

```

1 1. Load + inspect

```

df_path <- latest_parquet(dir = "../data/processed")
cat("loading:", df_path, "\n")

```

loading: ../data/processed/conversion_v2_20260502T235751Z.parquet

```

df_raw <- arrow::read_parquet(df_path) |> as_tibble()
cat("shape:", nrow(df_raw), "x", ncol(df_raw), "\n")

```

shape: 305 x 93

```

df_raw |>
  count(source, label) |>
  arrange(source, label) |>
  knitr::kable(caption = "Source x label cross-tab")

```

Table 1: Source × label cross-tab

source	label	n
checkout_session	0	279
checkout_session	1	7
payment_intent	0	16
payment_intent	1	3

```

miss_pct <- df_raw |>
  summarise(across(everything(), ~ mean(is.na(.x)))) |>
  pivot_longer(everything(), names_to = "column", values_to = "pct_missing") |>
  arrange(desc(pct_missing))

miss_pct |>
  filter(pct_missing > 0) |>
  head(30) |>
  mutate(pct_missing = scales::percent(pct_missing, accuracy = 0.1)) |>
  knitr::kable(caption = "Top-30 columns by % missing")

```

Table 2: Top-30 columns by % missing

column	pct_missing
chg_failure_code	100.0%
chg_failure_message	100.0%
apollo_title	100.0%
apollo_seniority	100.0%
apollo_org_industry	100.0%
cs_cart_n_lines	100.0%
cs_cart_total_cents	100.0%
cs_fulfillment_mode	100.0%
cs_recovery0_at	100.0%
cs_recovery3_at	100.0%
cs_sms_sent	100.0%
gads_lifetime_impressions	100.0%
gads_lifetime_clicks	100.0%
gads_lifetime_spend_usd	100.0%
gads_lifetime_conversions	100.0%
gads_lifetime_ctr_avg	100.0%
gads_lifetime_cpc_avg	100.0%
ga4_device_category	99.7%
ga4_device_browser	99.7%
ga4_geo_city	99.7%
ga4_traffic_source	99.7%
ga4_traffic_medium	99.7%
ga4_event_count	99.7%
ga4_pageviews	99.7%
ga4_scrolls	99.7%
ga4_form_starts	99.7%
fbclid	99.0%
metadata_utm_source	98.7%
metadata_utm_medium	98.7%
metadata_utm_campaign	98.7%

```
df_raw |>
  group_by(email_hash) |>
  summarise(
    attempts = n(),
    eventually_paid = max(as.integer(label)),
    .groups = "drop"
  ) |>
  count(attempts, eventually_paid) |>
  arrange(desc(attempts)) |>
  head(15) |>
  knitr::kable(caption = "Attempts-per-email distribution (top 15 buckets)")
```

Table 3: Attempts-per-email distribution (top 15 buckets)

attempts	eventually_paid	n
280	0	1
6	1	1

attempts	eventually_paid	n
3	1	1
2	0	2
1	0	6
1	1	6

Outlier flag: the bucket showing several hundred attempts under a single `email_hash` is the anonymous (NA) bucket — one or more bots/probes sharing a null email. We cap that in step 2 below.

2 2. Clean (drop leakage, cap outlier, type-coerce)

```
df_clean <- clean_attempts(df_raw, max_per_anon_bucket = 3, verbose = TRUE)
```

```
[clean] capped anonymous attempts: 305 -> 29 rows
```

```
[clean] dropped 17 leakage/ID columns
```

```
cat("after clean:", nrow(df_clean), "rows x", ncol(df_clean), "cols\n")
```

```
after clean: 29 rows x 76 cols
```

```
df_clean |> count(label) |> knitr::kable(caption = "Class balance after cleaning")
```

Table 4: Class balance after cleaning

label	n
0	19
1	10

Important: capping anonymous attempts dramatically changes the class balance because most negatives in the raw data are anonymous bot/probe traffic. We move from ~3% positive (raw) to ~34% positive (cleaned). This is a deliberate trade-off — the model now learns “what predicts conversion among real-looking attempts” instead of “is this anonymous traffic.” Bump `max_per_anon_bucket` if you want to keep more of the original imbalance.

3 3. Recipes

```
rec_balanced <- build_recipe(df_clean, use_smote = FALSE)
rec_smote     <- build_recipe(df_clean, use_smote = TRUE)

# Sanity-check the prepped feature matrix
prep_balanced <- prep(rec_balanced)
baked <- bake(prep_balanced, new_data = NULL)
cat("recipe-prepped:", nrow(baked), "x", ncol(baked), "cols\n")
```

recipe-prepped: 29 x 73 cols

```
cat("predictor cols (first 25):\n")
```

predictor cols (first 25):

```
cat(paste(" -", head(setdiff(names(baked), "label"), 25)), sep = "\n")
```

```
- amount_cents
- attempt_n_lifetime
- prior_successes_lifetime
- prior_failures_lifetime
- is_first_attempt
- prior_attempts_lifetime
- seconds_since_prior_attempt
- created_dow
- created_hour_utc
- mc_avg_open_rate
- mc_avg_click_rate
- mc_tag_count
- mc_in_audience
- resend_send_count
- resend_unique_types
- zip_median_income
- zip_population
- zip_median_age
- zip_household_size
- zip_pct_black
- zip_pct_white
- zip_pct_hispanic
- has_meta_campaign_match
- has_fbclid
- has_gclid
```

4 4. Models

For the **balanced** run we add `scale_pos_weight = neg/pos` to XGBoost and `class.weights` to ranger. glmnet (logistic) gets observation `case_weights` via the workflow. SMOTE versions skip these and let synthetic resampling balance the recipe output.

```
n_pos <- sum(df_clean$label == "1")
n_neg <- sum(df_clean$label == "0")
pos_weight <- n_neg / max(n_pos, 1)
cat("n_pos:", n_pos, " n_neg:", n_neg, " pos_weight:", round(pos_weight, 2), "\n")
```

n_pos: 10 n_neg: 19 pos_weight: 1.9

```

spec_lr <- logistic_reg(penalty = 0.01, mixture = 1) |>
  set_engine("glmnet") |>
  set_mode("classification")

spec_rf_balanced <- rand_forest(trees = 500, min_n = 5) |>
  set_engine(
    "ranger",
    importance = "permutation",
    class.weights = c("0" = 1, "1" = pos_weight)
  ) |>
  set_mode("classification")

spec_rf_smote <- rand_forest(trees = 500, min_n = 5) |>
  set_engine("ranger", importance = "permutation") |>
  set_mode("classification")

spec_xgb_balanced <- boost_tree(
  trees = 500,
  learn_rate = 0.05,
  tree_depth = 4
) |>
  set_engine("xgboost", scale_pos_weight = pos_weight) |>
  set_mode("classification")

spec_xgb_smote <- boost_tree(
  trees = 500,
  learn_rate = 0.05,
  tree_depth = 4
) |>
  set_engine("xgboost") |>
  set_mode("classification")

```

5 5. Cross-validation + fit

```

folds <- vfold_cv(df_clean, v = 5, strata = label)

metrics <- metric_set(yardstick::pr_auc, yardstick::roc_auc, yardstick::brier_class)

ctrl <- control_resamples(save_pred = TRUE, allow_par = FALSE)

```

5.1 5a. Manual per-model fits

Fitting each (model × resampling) combo individually so we can see exactly which combo fails (workflow_set hides this). Six fits total: 3 models × 2 resampling strategies.

```

safe_fit <- function(spec, rec, folds, metrics, ctrl) {
  wf <- workflow() |> add_recipe(rec) |> add_model(spec)
  res <- tryCatch(
    fit_resamples(wf, resamples = folds, metrics = metrics, control = ctrl),

```

```

    error = function(e) {
      cat("[fit FAILED]:", conditionMessage(e), "\n")
      NULL
    }
  )
  res
}

run_specs <- list(
  list(model = "lr", resampling = "balanced", spec = spec_lr, rec = rec_balanced),
  list(model = "rf", resampling = "balanced", spec = spec_rf_balanced, rec = rec_balanced),
  list(model = "xgb", resampling = "balanced", spec = spec_xgb_balanced, rec = rec_balanced),
  list(model = "lr", resampling = "smote", spec = spec_lr, rec = rec_smote),
  list(model = "rf", resampling = "smote", spec = spec_rf_smote, rec = rec_smote),
  list(model = "xgb", resampling = "smote", spec = spec_xgb_smote, rec = rec_smote)
)

results_list <- lapply(run_specs, function(s) {
  cat("- fitting:", s$model, s$resampling, "... \n")
  res <- safe_fit(s$spec, s$rec, folds, metrics, ctrl)
  list(model = s$model, resampling = s$resampling, result = res)
})

```

```

- fitting: lr balanced ...

- fitting: rf balanced ...

- fitting: xgb balanced ...

- fitting: lr smote ...

- fitting: rf smote ...

- fitting: xgb smote ...

```

6 6. Headline metrics table

```

collect_one <- function(item) {
  na_row <- tibble(model = item$model, resampling = item$resampling,
    .metric = c("pr_auc", "roc_auc", "brier_class"),
    mean = NA_real_, std_err = NA_real_, n = 0L)
  if (is.null(item$result)) return(na_row)
  m <- tryCatch(collect_metrics(item$result), error = function(e) NULL)
  if (is.null(m) || nrow(m) == 0) {
    cat("[collect FAILED]:", item$model, item$resampling,
      "\n- all folds errored. Run show_notes() on item$result to see why.\n")
    return(na_row)
  }
  m |>

```

```

dplyr::filter(.metric %in% c("pr_auc", "roc_auc", "brier_class")) |>
dplyr::select(.metric, mean, std_err, n) |>
dplyr::mutate(model = item$model, resampling = item$resampling, .before = 1)
}

headline_full <- purrr::map_dfr(results_list, collect_one) |>
  pivot_wider(
    names_from = .metric,
    values_from = c(mean, std_err)
  ) |>
  dplyr::select(
    model, resampling,
    pr_auc = mean_pr_auc, pr_auc_se = std_err_pr_auc,
    roc_auc = mean_roc_auc, roc_auc_se = std_err_roc_auc,
    brier = mean_brier_class
  ) |>
  dplyr::arrange(dplyr::desc(pr_auc))

floor_pr_auc <- mean(df_clean$label == "1")

cat("majority-baseline PR-AUC floor (= class prior):",
    round(floor_pr_auc, 4), "\n\n")

```

majority-baseline PR-AUC floor (= class prior): 0.3448

```

headline_full |>
  dplyr::mutate(dplyr::across(where(is.numeric), ~ round(.x, 3))) |>
  knitr::kable(caption = "Headline metrics: 5-fold stratified CV")

```

Table 5: Headline metrics: 5-fold stratified CV

model	resampling	pr_auc	pr_auc_se	roc_auc	roc_auc_se	brier
xgb	smote	0.879	0.079	0.825	0.094	0.206
lr	balanced	0.851	0.047	0.717	0.089	0.222
lr	smote	0.851	0.047	0.717	0.089	0.237
rf	smote	0.824	0.072	0.742	0.086	0.173
xgb	balanced	0.796	0.069	0.708	0.083	0.182
rf	balanced	0.786	0.087	0.717	0.097	0.191

7 7. Variable importance

```

fit_for_importance <- function(spec, rec) {
  workflow() |>
  add_recipe(rec) |>
  add_model(spec) |>
  fit(data = df_clean)
}

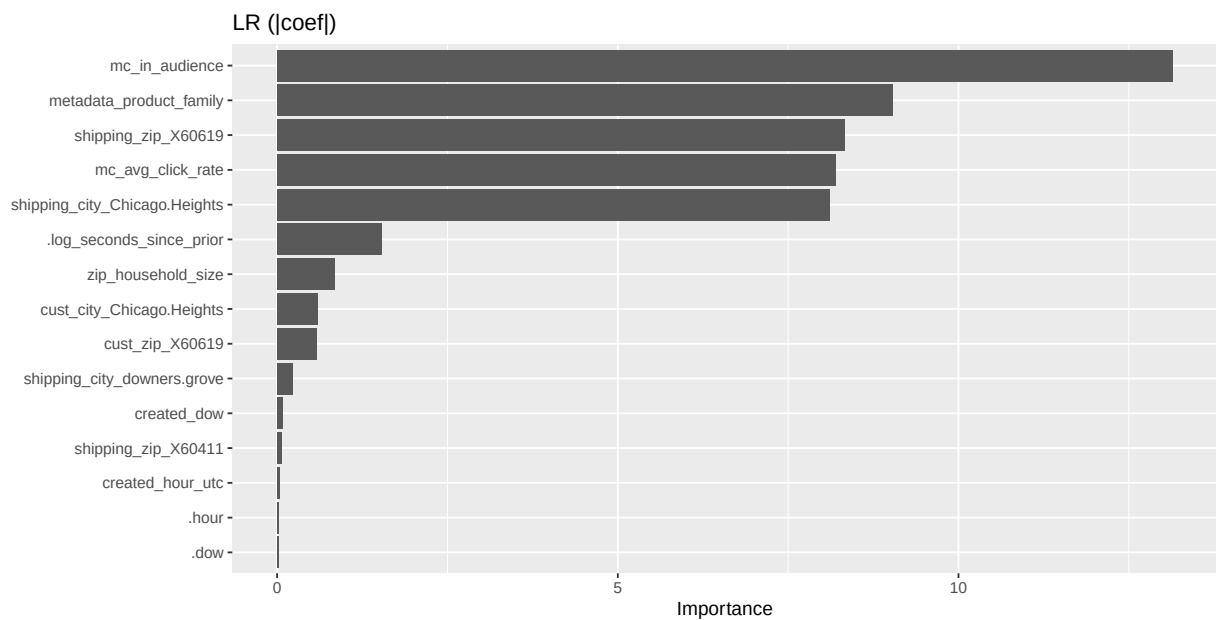
```



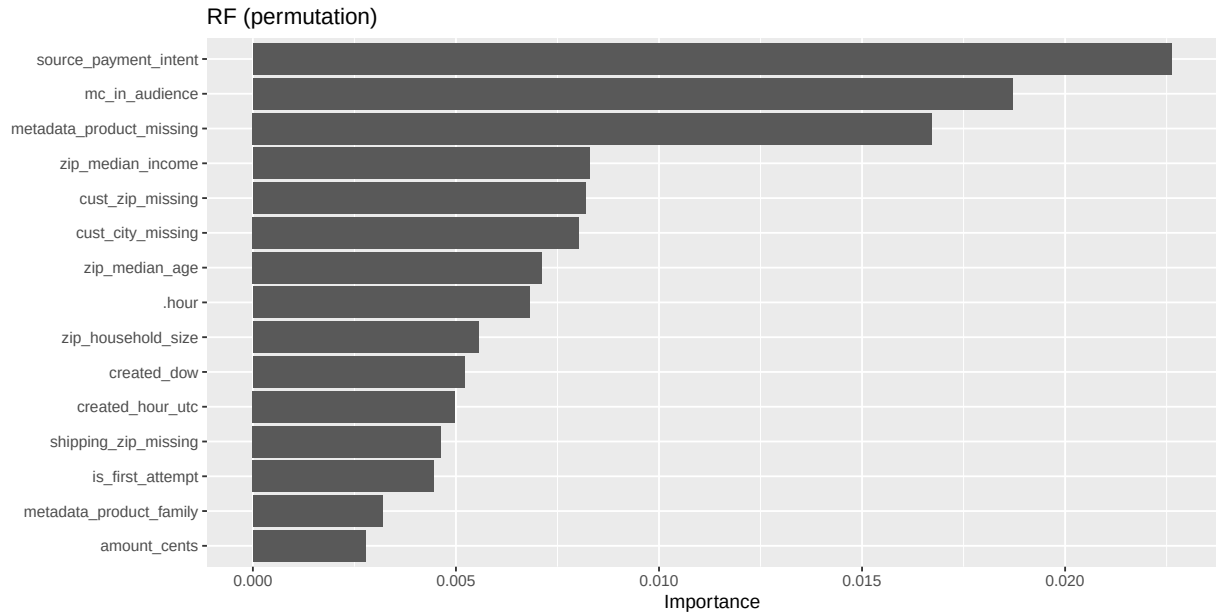
```
# Use the BALANCED variants for importance to keep apples-to-apples; we'll
# just look at which features each model thinks matter most.
fit_lr <- fit_for_importance(spec_lr,          rec_balanced)
fit_rf <- fit_for_importance(spec_rf_balanced, rec_balanced)
fit_xgb <- fit_for_importance(spec_xgb_balanced, rec_balanced)
```

```
p_lr <- vip(extract_fit_engine(fit_lr), num_features = 15) + ggtitle("LR (|coef|)")
p_rf <- vip(extract_fit_engine(fit_rf), num_features = 15) + ggtitle("RF (permutation)")
p_xgb <- vip(extract_fit_engine(fit_xgb), num_features = 15) + ggtitle("XGBoost (gain)")

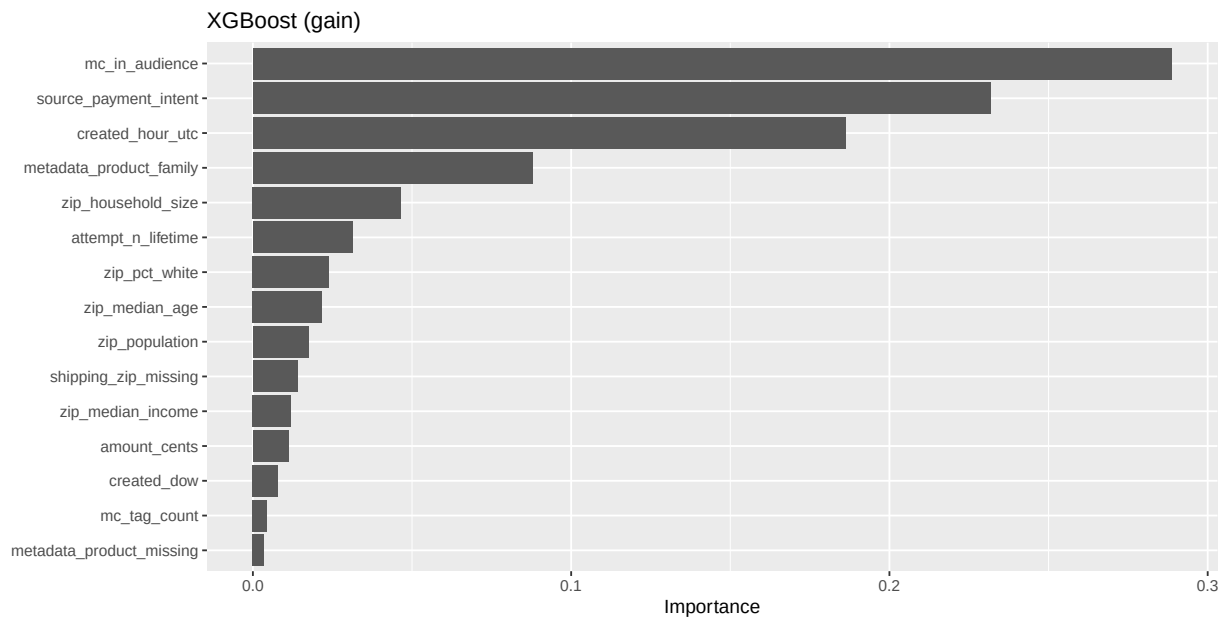
print(p_lr)
```



```
print(p_rf)
```



```
print(p_xgb)
```



8 8. Summary

```
best_row <- headline_full |> slice_max(pr_auc, n = 1)

# Consensus top features across the three models (intersect of top-10 by importance)
top_features_one <- function(fit, n = 10) {
  vi <- vi(extract_fit_engine(fit))
```

```

  if (nrow(vi) == 0) return(character(0))
  vi |> arrange(desc(abs(Importance))) |> head(n) |> pull(Variable)
}

top_lr <- top_features_one(fit_lr, 10)
top_rf <- top_features_one(fit_rf, 10)
top_xgb <- top_features_one(fit_xgb, 10)

consensus <- Reduce(intersect, list(top_lr, top_rf, top_xgb))
two_of_three <- unique(c(
  intersect(top_lr, top_rf),
  intersect(top_lr, top_xgb),
  intersect(top_rf, top_xgb)
))

cat("### Best model\n\n")

```

8.1 Best model

```
cat("- ", best_row$model, " (", best_row$resampling, ")\n", sep = "")
```

- xgb (smote)

```
cat("- PR-AUC: ", round(best_row$pr_auc, 3),
    " ± ", round(best_row$pr_auc_se, 3),
    " (floor = ", round(floor_pr_auc, 3), ")\n", sep = "")
```

- PR-AUC: 0.879 ± 0.079 (floor = 0.345)

```
cat("- ROC-AUC: ", round(best_row$roc_auc, 3),
    " ± ", round(best_row$roc_auc_se, 3), "\n\n", sep = "")
```

- ROC-AUC: 0.825 ± 0.094

```
cat("### SMOTE verdict\n\n")
```

8.2 SMOTE verdict

```

balanced_best <- headline_full |> filter(resampling == "balanced") |>
  slice_max(pr_auc, n = 1)
smote_best <- headline_full |> filter(resampling == "smote") |>
  slice_max(pr_auc, n = 1)
delta <- smote_best$pr_auc - balanced_best$pr_auc
verdict <- if (is.na(delta)) "n/a" else if (delta > 0.02) "SMOTE helped" else if (delta < -0.02) "SMOTE
cat("- balanced best PR-AUC: ", round(balanced_best$pr_auc, 3), "\n", sep = "")

```

- balanced best PR-AUC: 0.851

```
cat("- SMOTE best PR-AUC: ", round(smote_best$pr_auc, 3), "\n", sep = "")
```

- SMOTE best PR-AUC: 0.879

```
cat("- delta: ", round(delta, 3), " → **", verdict, "**\n\n", sep = "")
```

- delta: 0.027 → **SMOTE helped**

```
cat("### Consensus top features (top-10 across LR, RF, XGB)\n\n")
```

8.3 Consensus top features (top-10 across LR, RF, XGB)

```
if (length(consensus) > 0) {
  for (f in consensus) cat("- ", f, "\n", sep = "")
} else {
  cat("- No features in all three top-10s. ",
      "2-of-3 agreement: ",
      paste(two_of_three, collapse = ", "), "\n", sep = "")
}
```

- mc_in_audience
- zip_household_size

```
cat("\n### Recommendations\n\n")
```

8.4 Recommendations

```
cat("- N=", nrow(df_clean), " with ", n_pos, " positives is too thin for a deployable model. ",
     "Treat these PR-AUC numbers as ±0.05 noise.\n", sep = "")
```

- N=29 with 10 positives is too thin for a deployable model. Treat these PR-AUC numbers as ±0.05 noise.

```
cat("- Note: the headline numbers reflect the **cleaned** dataset ",
     "(", nrow(df_clean), " rows, base rate = ", round(floor_pr_auc, 2),
     "). The raw spine was ", nrow(df_raw), " rows; ",
     nrow(df_raw) - nrow(df_clean), " were dropped as anonymous-bucket overflow.\n", sep = "")
```

- Note: the headline numbers reflect the **cleaned** dataset (29 rows, base rate = 0.34). The raw spine was 305 rows; 276 were dropped as anonymous-bucket overflow.

```
cat("- Re-run weekly. Custom-cart launch will inflate both row count AND ",
     "GA4 BQ join coverage (currently 1/305).\n")
```

- Re-run weekly. Custom-cart launch will inflate both row count AND GA4 BQ join coverage (currently 1/305).

```
cat("- Drop features that appear in ZERO importance plots.\n")
```

- Drop features that appear in ZERO importance plots.

```
cat("- Consider instrumenting: time-on-page (Clarity), scroll depth (GA4 once cart launches), ",  
      "ad-click-to-checkout latency (Meta CAPI).\n")
```

- Consider instrumenting: time-on-page (Clarity), scroll depth (GA4 once cart launches), ad-click-to-checkout latency (Meta CAPI).